

Chapter 6

Inter-Process Communication (IPC)

Mechanisms

Pipes, Message Queues, Shared Memory

Print Version of Lectures Notes of *Operating Systems*

Technical University of Cluj-Napoca (UTCN)
Computer Science Department

Adrian Coleșa

April 26th, 2016

6.1

Purpose and Contents

The purpose of this chapter

- Presents the main mechanisms for communication between processes
- Presents different examples of using them

6.2

Bibliography

- A. Colesă, I. Ignat, Z. Somodi, *Sisteme de Operare. Chestiuni Teoretice și Practice*, 2007, Chapters 8 and 11, p. 105 – 117, p. 149 – 169 (Romanian only — see the pdf file on moodle page). For the english version see the html pages also on moodle page at Lecture resources.

6.3

Contents

1	Pipe (FIFO Files)	2
1.1	Characteristics and Communication Principles	2
1.2	Examples	3
2	Message Queues	5
2.1	Characteristics and Communication Principles	5
2.2	Examples	5
3	Shared Memory	5
3.1	Characteristics and Communication Principles	5
3.2	Examples	6

6.4

IPC Mechanisms

- indirect communication
 - synchronization mechanisms, e.g. semaphores
- direct communication
 - wait/exit system calls

- pipes
- message queues
- shared memory

6.5

1 Pipe (FIFO Files)

1.1 Characteristics and Communication Principles

Characteristics of a pipe

- provided as file
 - accessed as a regular file (open, read, write)
- it is a *special file*
 - controlled by the operating system
 - exposed in a special format (FIFO)
 - OS imposes a special way to work with it
- it is an IPC mechanism
 - more processes (any number) can communicate through it

6.6

Communication Principles

- based on Producer/Consumer Synchronization Pattern
- data access
 - FIFO
 - there is no seek in the pipe
 - once read, data from pipe is no more available
 - circular fix-sized buffer → *no end of file*
- synchronization
 1. reading from an empty pipe blocks the calling processes (consumer)
 2. writing into a full pipe blocks the calling processes (producer)
- exceptions
 1. reading from an empty pipe with no writer connected returns immediately as if end of file was detected
 2. writing into a full pipe with no reader connected returns immediately with an error

6.7

Uni and Bi-directional Pipes

- unidirectional: used for a one-way communication
- bidirectional: used for both directions (N directions)
- normally a user convention, but sometime an OS support

6.8

Types of Pipes

- nameless (anonymous) pipes
- pipes with name (FIFO files)

6.9

Nameless (Anonymous) Pipes

- created with a special system call
- have no name
- do not occur in the file system
- not visible, but to their creator process
- automatically opened by the OS for the creator process
- used normally for communication between processes in parent-child relationship

6.10

Pipes With Name (FIFO Files)

- created by a special system call
- have a name visible in the file system
- any process that wants to use them must open them as any other file

6.11

1.2 Examples

Linux Anonymous Pipes

- system call

```
int pipe(int fd[2]);
```

- usage

```
int fd[2];
int nrl = 10, nr2;

pipe(fd);

if (fork() == 0) {
    close(fd[0]);
    write(fd[1], &nrl, sizeof(int));
} else {
    close(fd[1]);
    read(fd[0], &nr2, sizeof(int));
}
```

6.12

Windows Anonymous Pipes

- Windows Win32 Function

```
BOOL WINAPI CreatePipe(
    __out PHANDLE hReadPipe,
    __out PHANDLE hWritePipe,
    __in_opt LPSECURITY_ATTRIBUTES lpPipeAttributes,
    __in DWORD nSize
);
```

- usage

```
HANDLE hReadPipe = NULL;
HANDLE hWritePipe = NULL;
DWORD dwRead, dwWritten;
CHAR chBuf[BUFSIZE];
SECURITY_ATTRIBUTES saAttr;
saAttr.nLength = sizeof(SECURITY_ATTRIBUTES);
saAttr.bInheritHandle = TRUE;
saAttr.lpSecurityDescriptor = NULL;

CreatePipe(&hReadPipe, &hWritePipe, &saAttr, 0);

WriteFile(hWritePipe, chBuf, 10, &dwWritten, NULL);
ReadFile(hReadPipe, chBuf, 10, &dwRead, NULL);
```

6.13

Linux FIFO Files

- system call

```
int mkfifo(char *name, mode_t permissions);
```

- usage

```
int fdPipe;
int nrl = 10, nr2;

mkfifo("FIFO", 0644);
fdPipe = open("FIFO", O_RDWR);

write(fdPipe, &nrl, sizeof(int));
read(fdPipe, &nr2, sizeof(int));
```

6.14

Windows FIFO Files. Server Process

- system call

```
CreateNamedPipe();
ConnectNamedPipe(hPipe, NULL);
```

- usage

```

hPipe = CreateNamedPipe(
    lpzPipename,          // pipe name
    PIPE_ACCESS_DUPLEX,   // read/write access
    PIPE_TYPE_MESSAGE |   // message type pipe
    PIPE_READMODE_MESSAGE | // message-read mode
    PIPE_WAIT,            // blocking mode
    PIPE_UNLIMITED_INSTANCES, // max. instances
    BUFSIZE,              // output buffer size
    BUFSIZE,              // input buffer size
    0,                    // client time-out
    NULL);                // default security attribute

fConnected = ConnectNamedPipe(hPipe, NULL);

ReadFile(
    hPipe,                // handle to pipe
    pchRequest,           // buffer to receive data
    BUFSIZE*sizeof(TCHAR), // size of buffer
    &cbBytesRead,          // number of bytes read
    NULL);                // not overlapped I/O

```

6.15

Windows FIFO Files. Client Process

```

LPTSTR lpzPipename = TEXT("\\\\.\\pipe\\mynamedpipe");

hPipe = CreateFile(
    lpzPipename, // pipe name
    GENERIC_READ | // read and write access
    GENERIC_WRITE,
    0,            // no sharing
    NULL,         // default security attributes
    OPEN_EXISTING, // opens existing pipe
    0,            // default attributes
    NULL);

WriteFile(
    hPipe,          // pipe handle
    lpvMessage,     // message
    (strlen(lpvMessage)+1)*sizeof(TCHAR), // message length
    &cbWritten,      // bytes written
    NULL);          // not overlapped

```

6.16

Linux Bidirectional Pipes

- first process

```

mkfifo("FIFO1", 0600);
mkfifo("FIFO2", 0600);
int fdW = open("FIFO1", O_WRONLY);
int fdR = open("FIFO2", O_RDONLY);

write(fdW, "1", 1);
read(fdR, &c, 1);

```

- second process

```

int fdR = open("FIFO1", O_RDONLY);
int fdW = open("FIFO2", O_WRONLY);

read(fdR, &c, 1);
write(fdW, "2", 1);

```

6.17

Command Line Pipe

- command line syntax

```
cmd_1 | cmd_2 | ... | cmd_n
```

- command interpreter (shell) code

```

int fd[n][2];
char cmd[n][256];
char argv[n][256][256];

for (i=0; i<n; i++) {
    if (i < n-1)
        pipe(fd[i]);

    if (fork() == 0) {
        if (i > 0) {
            close(fd[i-1][1]);
            dup2(fd[i-1][0], 0);
            close(fd[i-1][0]);
        }

        if (i < n-1) {
            close(fd[i][0]);
            dup2(fd[i][1], 1);
            close(fd[i][1]);
        }
        execvp(cmd[i], argv[i]);
    }
    else {
        close(fd[i][0]);
        close(fd[i][1]);
    }
}

```

6.18

2 Message Queues

2.1 Characteristics and Communication Principles

Characteristics

- a more specialized pipe
- group bytes in messages, making distinction between messages
- different types (labels) of messages
- processes can get messages of specified type from queue

6.19

Communication Principles

- same as pipe, though sometimes FIFO order can be broken, when a message of a specified type is requested

6.20

2.2 Examples

Two-Way Communication

- first process

```
struct msg {
    long type;
    int data;
} msg1, msg2;

int msgId = msgget(10000, IPC_CREAT | 0600);
msg1.type = 1;
msg1.data = getpid();

// send a message of type 1
msgsnd(msgId, &msg1, sizeof(msg1) - sizeof(long), 0);

// gets a message of type 1
msgrcv(msgId, &msg2, sizeof(msg1) - sizeof(long), 2, 0);
```

- second process

```
struct msg {
    long type;
    int data;
} msg1, msg2;

int msgId = msgget(10000, 0);
msg2.type = 2;
msg2.data = getpid();

// gets a message of type 1
msgrcv(msgId, &msg2, sizeof(msg1) - sizeof(long), 1, 0);

// send a message of type 2
msgsnd(msgId, &msg1, sizeof(msg1) - sizeof(long), 0);
```

6.21

3 Shared Memory

3.1 Characteristics and Communication Principles

Characteristics

- a common (shared) memory area belonging to more process address spaces
- created with a special system call
- managed by the OS
- the fastest IPC mechanism

6.22

Principles

- no need for special system calls
- used as any process memory area referenced by a pointer
- sequence of bytes, whose structure is known by the collaborating processes
- the concurrent accesses to it is NOT synchronized by the OS $\Rightarrow$ the processes SHOULD make this

6.23

3.2 Examples

Two-Way Communication

- first process

```
int shmId = shmget(10000, sizeof(int), IPC_CREAT | 0600);  
int *i = shmat(shmId, 0, 0);  
*i = 10; // write shared memory -> "send" information
```

- second process

```
int shmId = shmget(10000, 0, 0);  
int *j = shmat(shmId, 0, 0);  
printf("Received_info_is_%d\n", *j);  
// read the shared memory -> "receive" information
```

6.24